Statistical Machine Learning Course Project Report

How to Run
1. Install packages: python -m pip install -r requirements.txt
2. Run Part 1: python run.py part1
3. Run Part 2 on Kaggle/Colab GPU if possible: python run.py part2 --encoder microsoft/deberta-v3-base
4. Optionally compare cached encoders: python run.py compare-encoders
5. Run Part 3 after embeddings are cached: python run.py part3 --encoder microsoft/deberta-v3-base

Introduction and Dataset used

This project studies multi-class text classification in a strictly pre-deep learning setting. The goal is to compare classical machine learning models trained on two different feature representations: sparse lexical features and frozen contextual embeddings. The learned models remain classical ML models, while the pretrained language model is used only as a feature extractor, which keeps the work aligned with the project brief.

The dataset used is Hugging Face ag_news. A fixed stratified sample of 8,000 documents was created with four classes: World, Sports, Business, and Sci/Tech. This dataset was chosen because it is valid under the brief, balanced, and suitable for comparing keyword-heavy sparse methods against contextual embedding methods.

From a statistical learning perspective, this project is useful because the same classification problem can be viewed through two different feature-design philosophies. The first treats documents as sparse token-count vectors, while the second represents them through a fixed embedding space obtained from a frozen pretrained encoder. By holding the downstream models classical, the comparison isolates the effect of representation.

Dataset Description and EDA

Class	Samples
World	2000
Sports	2000
Business	2000
Sci/Tech	2000

The dataset was split into 6,400 training samples and 1,600 test samples using stratified sampling. The average text length is 38.03 tokens, the median is 37, the shortest sample has 10 tokens, and the longest has 139 tokens.

These summary statistics indicate that AG News consists of short news snippets rather than long-form articles. That makes the task especially interesting: short documents often contain strong topical keywords, which helps sparse models, but they also leave less room for context recovery, which is where dense encoder embeddings can help.

- Class distribution plot
- Text length histogram

- Model comparison chart
- Confusion matrices for the best sparse and contextual models
- Explained variance plot for SVD and PCA

The class balance is especially useful for evaluation because Accuracy and Macro F1 can be interpreted without worrying about strong label skew. This means that differences between models are more likely to reflect the quality of feature extraction and decision boundaries rather than imbalance effects.

The text length histogram also shows moderate variation rather than extreme outliers. This matters because both sparse vectorizers and fixed-length transformer encoding benefit from documents that are not excessively long; truncation and rare-token issues become less severe than they would on full news articles.

Methodology

Two sparse feature spaces were constructed: unigram Bag-of-Words and bigram TF-IDF. Both pipelines apply lowercasing, stop-word removal, tokenization through the vectorizer, and vocabulary pruning using min_df, max_df, and max_features.

For contextual features, an encoder-only transformer is used strictly as a frozen feature extractor. The implementation now supports several 768-dimensional encoders and defaults to DeBERTa-v3 Base as the strongest recommended candidate. Two representations are extracted: CLS embeddings and mean-pooled embeddings. Embeddings are computed once and cached in NPZ format.

The upgraded workflow separates Part 1 and Part 2 instead of blending them into one run. Part 1 focuses only on sparse feature engineering and classical ML. Part 2 focuses only on encoder embedding extraction, ideally on Kaggle or Colab GPUs. A dedicated comparison stage can rank all cached encoders, and Part 3 then consumes the selected embeddings for classical ML training and evaluation.

The Bag-of-Words representation captures direct token frequency and is highly interpretable. The bigram TF-IDF representation extends this by encoding important two-word phrases and down-weighting globally common tokens. This makes TF-IDF better suited for distinguishing categories whose semantics are partly expressed through short phrases.

The contextual representation uses the selected encoder only as a feature extractor, not as a trainable classifier. The CLS vector provides a single summary token representation, while mean pooling averages information across all contextualized token states. Comparing these two variants helps determine whether global sentence meaning or dedicated classification-token behavior is more useful for this task.

Batch inference is used during encoder extraction, and tqdm progress bars are included so long runs are visible and manageable. This is especially helpful when Part 2 is run on a remote GPU notebook environment.

- Logistic Regression
- Linear SVM
- RBF SVM
- KNN
- KMeans

KMeans was adapted into a simple classifier by assigning each cluster the majority class from the training split. The main evaluation metrics are Accuracy and Macro F1-score.

Logistic Regression and Linear SVM were chosen because they are strong linear baselines on high-dimensional text features. RBF SVM was included to test whether non-linear decision boundaries help once the representation is more compact or semantically rich. KNN serves as a distance-based baseline, while KMeans provides an unsupervised reference point.

The implementation also emphasizes efficiency. Sparse features are cached, contextual embeddings are cached, and the final workflow is wrapped in a single launcher so that the project can be rerun without manually reconstructing the pipeline each time.

Experiments

1. Load the ag_news dataset and create a fixed 8,000-sample subset.
2. Split the data into training and test sets using stratified sampling.
3. Build Bag-of-Words and bigram TF-IDF sparse features.
4. Extract CLS and mean-pooled 768-dimensional encoder embeddings and store them in NPZ format.
5. Train the required classical models on both feature spaces.
6. Evaluate using Accuracy and Macro F1-score.
7. Apply Truncated SVD to TF-IDF and PCA to encoder embeddings.
8. Analyze explained variance and compare reduced-space performance.

For dimensionality reduction, Truncated SVD was used for TF-IDF because it works directly on sparse matrices. PCA was used for the dense contextual embeddings.

The experiment design intentionally compares full and reduced feature spaces. This helps answer two practical questions: whether compression causes a major drop in classification quality, and whether the richer contextual space is inherently more compact than the sparse lexical space.

For the sparse pipeline, a small amount of model selection was used where it provided good trade-offs, while the dense contextual pipeline was kept simpler to improve stability and reproducibility. This reflects a realistic engineering decision: on small to medium NLP tasks, moderate tuning often gives most of the gain, while heavy search can add cost without proportional benefit.

The project outputs figures, tables, and confusion matrices automatically. This makes the workflow suitable not only for a one-time report, but also for repeated comparison if more models or alternative encoders are tested later.

To improve creativity in the visual analysis, the project now includes t-SNE projections for sparse reduced features and encoder embeddings. These visualizations make it easier to judge whether class separation is naturally emerging in the learned representation space.

Results

Feature Space	Best Model	Accuracy	Macro F1
Unigram Bag-of-Words	RBF SVM	0.8706	0.8700
Bigram TF-IDF	RBF SVM	0.8831	0.8830
Frozen encoder CLS (baseline run)	RBF SVM	0.8844	0.8843
Frozen encoder Mean (baseline run)	RBF SVM	0.8969	0.8969

In the current local benchmark, the best overall model was a mean-pooled frozen encoder baseline with an RBF SVM, achieving 0.8969 Accuracy and 0.8969 Macro F1. The best sparse model was bigram TF-IDF with an RBF SVM, achieving 0.8831 Accuracy and 0.8830 Macro F1. The upgraded codebase is organized so stronger 768-dimensional encoders such as DeBERTa-v3 Base can be extracted separately on Kaggle or Colab GPU and then reused in Part 3.

The strongest class in both the sparse and contextual settings was Sports, while Business and Sci/Tech were relatively harder. This is reasonable because those two classes often share company names,

product references, and market-related vocabulary.

Model	World	Sports	Business	Sci/Tech
Best sparse F1	0.902	0.949	0.844	0.864
Best contextual F1	0.901	0.966	0.862	0.879

Reduced Feature Space	Best Model	Accuracy	Macro F1
TF-IDF + SVD (300)	RBF SVM	0.8831	0.8829
Embeddings + PCA (100)	RBF SVM	0.8919	0.8917

The explained variance results show that TF-IDF with 300 SVD components retained 0.2659 cumulative explained variance, while the 100-component PCA representation of mean-pooled embeddings retained about 0.8510.

This is a useful contrast. Even though the sparse TF-IDF system remains competitive, its information is spread across a very large basis. The contextual embedding space, in contrast, keeps much more of its predictive structure even after a stronger dimensionality reduction step.

Another important observation is that linear methods remained surprisingly strong in both sparse and contextual settings. The best contextual model was non-linear, but Logistic Regression and Linear SVM stayed close behind, especially on the mean-pooled embedding representation.

Project Files and Implementation Roles

For completeness and reproducibility, the project folder contains a small set of top-level files and a code directory. Each file has a specific role in the workflow rather than being included as redundant support material.

- report.pdf: the final written report submitted for the course project.
- requirements.txt: the Python package list needed to run the project on a new machine.
- run.py: the executable entry point with argparse-style stage selection for Part 1, Part 2, Part 3, and encoder comparison.

The code directory contains the main implementation files. Their roles are summarized below.

- config.py: stores project-wide constants such as dataset size, random seed, embedding model name, and directory paths.
- data.py: downloads, samples, labels, and splits the AG News dataset into training and test sets.
- encoder_registry.py: lists the supported 768-dimensional encoder-only models and the recommended default.
- sparse_features.py: builds the unigram Bag-of-Words and bigram TF-IDF pipelines and caches the sparse matrices.
- embeddings.py: extracts frozen encoder CLS and mean-pooled embeddings, performs batched inference, validates cached arrays, and stores them in NPZ format.
- models.py: defines the classical learning models, dimensionality reduction utilities, and limited tuning logic.
- evaluation.py: computes Accuracy, Macro F1, confusion matrices, and structured result tables.
- plots.py: generates the visual outputs such as class distribution, comparison charts, confusion heatmaps, and t-SNE graphs.
- part1_sparse.py: runs only the sparse-feature stage required for Part 1.

- part2_embeddings.py: runs only the encoder embedding extraction stage required for Part 2, ideally on Kaggle or Colab GPU.
- part3_contextual.py: trains classical ML models using the cached encoder embeddings from Part 2.
- encoder_comparison.py: evaluates all cached encoders and writes a leaderboard to help select the most effective one.
- cli.py: stage dispatcher used by the executable file.
- run_experiments.py: shared orchestration helpers reused by the modular parts.
- generate_clean_report_pdf.py: rebuilds the final PDF report in a clean layout.

This separation makes the project easier to understand. Data handling, feature engineering, model training, evaluation, visualization, and reporting are all modularized, which improves both readability and maintainability.

Discussion

The results confirm that sparse lexical features remain strong for short news classification, especially when using TF-IDF with bigrams. However, the contextual embedding representation provided the best final performance, showing that semantic information still offers a measurable advantage.

Among the contextual variants, mean pooling performed better than CLS pooling. This suggests that for short text classification, aggregating token information across the whole sequence produces a more robust representation than relying only on the CLS token.

Dimensionality reduction preserved a large portion of embedding performance, which indicates that the dense semantic representation is relatively compact. By contrast, sparse TF-IDF needed a higher-dimensional basis to stay competitive.

The reduced embedding model remained very close to the best unreduced contextual system, which suggests that most of the task-relevant signal was concentrated in a smaller subspace. This makes the embedding representation attractive not only for accuracy, but also for compression.

From the standpoint of classical machine learning, the most interesting lesson is that the downstream classifier family is not the whole story. Feature quality strongly shapes the apparent usefulness of linear versus non-linear models. A weak feature space can make a model look underpowered, while a strong representation can make even simple classifiers highly competitive.

The weaker KMeans and KNN results also reinforce an important point. Not every classical method is equally well suited to high-dimensional NLP data. Distance-based and clustering methods can still be informative as baselines, but supervised discriminative methods remain better choices when labels are available.

There is also a practical reproducibility lesson. A full project should not only maximize performance, but also be easy to rerun. That is why this implementation includes caching and a single launcher script. In real ML workflows, reproducibility and usability are often as important as a small gain in validation score.

Conclusion and Learnings

This project shows that strong NLP classification performance can be achieved using classical machine learning when the feature representation is carefully designed. Bigram TF-IDF provides a strong, interpretable baseline, while frozen encoder embeddings offer the best overall performance without violating the project constraint against fine-tuning.

The main learning is that feature representation matters at least as much as classifier choice. Sparse methods are efficient and transparent, whereas contextual embeddings are more expressive and compact under dimensionality reduction.

Overall, the project supports a balanced conclusion. Classical sparse NLP techniques are still very effective for topic classification, especially on short text. At the same time, frozen contextual embeddings offer a practical bridge to modern representation learning and can improve performance even when no deep classifier is trained.

If this work were extended further, the most natural next steps would be to benchmark all supported 768-dimensional encoders under the same frozen setup, evaluate a second Hugging Face dataset, and perform a deeper class-wise error analysis to understand where contextual semantics help the most.